STRUCTURES IN C

1. What is a Structure?

A **structure (`struct`)** in C is a user-defined data type that allows grouping variables of different data types under one name.

Unlike arrays (which store same data type), structures store **heterogeneous data**.

2. Why Use Structures?

- To represent real-world entities (Student, Employee, Book, etc.)
- To group related data
- To improve code organization
- To pass multiple values to functions

3. Syntax of Structure

```
struct structure_name {
    data_type member1;
    data_type member2;
    data_type member3;
};
```

Example:

```
struct Student {
    int roll;
    char name[50];
    float marks;
};
```

4. Declaring Structure Variables

After defining the structure:

```
struct Student s1, s2;
```

OR

```
struct Student {
    int roll;
    char name[50];
    float marks;
} s1, s2;
```

5. Accessing Structure Members

Using **dot operator (.)**

```
s1.roll = 101;
strcpy(s1.name, "Rahul");
s1.marks = 89.5;
```

6. Initialization of Structure

```
struct Student s1 = {101, "Rahul", 89.5};
```

7. Array of Structures

```
struct Student s[3];
```

Access:

```
s[0].roll = 1;
```

Used when handling multiple records.

8. Nested Structures

Structure inside another structure.

```
struct Date {
    int day, month, year;
};

struct Student {
    int roll;
    char name[50];
    struct Date dob;
};
```

Access:

```
s1.dob.day = 12;
```

9. Pointer to Structure

```
struct Student s1;
struct Student *ptr = &s1;
```

Access using arrow operator:

```
ptr->roll = 101;
```

10. Passing Structure to Functions

a) Pass by Value

```
void display(struct Student s);
```

b) Pass by Reference (Efficient)

```
void display(struct Student *s);
```

11. typedef with Structure

To simplify syntax:

```
typedef struct {
    int roll;
    char name[50];
    float marks;
} Student;
```

Now you can write:

```
Student s1;
```

12. Memory Allocation in Structures

- Memory = Sum of sizes of members
- Padding may occur for alignment
- Use `sizeof(struct_name)`

Example:

```
printf("%lu", sizeof(struct Student));
```

◆ C Program Using Structure (Student Management Example)

```
#include <stdio.h>
#include <string.h>

struct Student {
    int roll;
    char name[50];
    float marks;
};

int main() {
    struct Student s;
```

```
    // Input
    printf("Enter Roll Number: ");
    scanf("%d", &s.roll);

    printf("Enter Name: ");
    scanf("%s", s.name);

    printf("Enter Marks: ");
    scanf("%f", &s.marks);

    // Output
    printf("\n--- Student Details ---\n");
    printf("Roll Number: %d\n", s.roll);
    printf("Name: %s\n", s.name);
    printf("Marks: %.2f\n", s.marks);

    return 0;
}
```

◆ Output Example

```
Enter Roll Number: 101
Enter Name: Rahul
Enter Marks: 88.5

--- Student Details ---
Roll Number: 101
Name: Rahul
Marks: 88.50
```

Array of Structures (Multiple Netflix Shows)

Netflix stores many shows, so we use **array of structures**.

```
#include <stdio.h>

struct NetflixShow {
    char title[100];
    char genre[50];
    float rating;
    int duration;
    int releaseYear;
};

int main() {
    struct NetflixShow shows[3];
    int i;
```

```
    for(i = 0; i < 3; i++) {
        printf("\nEnter details for Show %d\n", i+1);

        printf("Title: ");
        scanf(" %[^\n]", shows[i].title);

        printf("Genre: ");
        scanf(" %[^\n]", shows[i].genre);

        printf("Rating: ");
        scanf("%f", &shows[i].rating);

        printf("Duration: ");
        scanf("%d", &shows[i].duration);

        printf("Release Year: ");
        scanf("%d", &shows[i].releaseYear);
    }

    printf("\n--- Netflix Shows List ---\n");

    for(i = 0; i < 3; i++) {
        printf("\nTitle: %s", shows[i].title);
        printf("\nGenre: %s", shows[i].genre);
        printf("\nRating: %.1f", shows[i].rating);
        printf("\nDuration: %d minutes", shows[i].duration);
        printf("\nRelease Year: %d\n", shows[i].releaseYear);
    }

    return 0;
}
```

Program 1: Netflix Program Using Structure

Conditions:

- Print movies released after 2010
- Find movie with **maximum rating**
- Find movie with **minimum rating**

(Example inspired by Netflix platform)

C Program

```
#include <stdio.h>

struct Movie {
    char title[50];
    int releaseYear;
    float rating;
};

int main() {
    int i;
    struct Movie m[3] = {
        {"Inception", 2010, 8.8},
        {"Interstellar", 2014, 8.6},
        {"The Irishman", 2019, 7.8}
    };

    float maxRating = m[0].rating;
    float minRating = m[0].rating;
    int maxIndex = 0, minIndex = 0;

    printf("Movies released after 2010:\n");

    for(i = 0; i < 3; i++) {
        if(m[i].releaseYear > 2010) {
            printf("%s (%d) - Rating: %.1f\n",
                   m[i].title,
                   m[i].releaseYear,
                   m[i].rating);
        }

        // Find maximum rating
        if(m[i].rating > maxRating) {
            maxRating = m[i].rating;
            maxIndex = i;
        }

        // Find minimum rating
        if(m[i].rating < minRating) {
            minRating = m[i].rating;
            minIndex = i;
        }
    }

    printf("\nMovie with Maximum Rating:\n");
    printf("%s - %.1f\n", m[maxIndex].title,
m[maxIndex].rating);

    printf("\nMovie with Minimum Rating:\n");
```

```
    printf("%s - %.1f\n", m[minIndex].title,
m[minIndex].rating);

    return 0;
}
```

Program 2: Student Details with Maximum Marks

Using:

- Structure
- Simple functions
- Find student with highest marks

C Program

```
#include <stdio.h>

struct Student {
    int roll;
    char name[50];
    float marks;
};

// Function to find and print student with maximum marks
void findMax(struct Student s[], int n) {
    int i, maxIndex = 0;

    for(i = 1; i < n; i++) {
        if(s[i].marks > s[maxIndex].marks) {
            maxIndex = i;
        }
    }

    printf("\nStudent with Maximum Marks:\n");
    printf("Roll No: %d\n", s[maxIndex].roll);
    printf("Name: %s\n", s[maxIndex].name);
    printf("Marks: %.2f\n", s[maxIndex].marks);
}

int main() {
    int i, n = 3;
    struct Student s[3];

    for(i = 0; i < n; i++) {
        printf("\nEnter details for Student %d\n", i+1);
```

```
        printf("Roll No: ");
        scanf("%d", &s[i].roll);

        printf("Name: ");
        scanf("%s", s[i].name);

        printf("Marks: ");
        scanf("%f", &s[i].marks);
    }

    findMax(s, n);

    return 0;
}
```